

Q1. Tower of Hanoi.

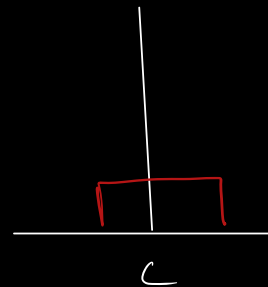
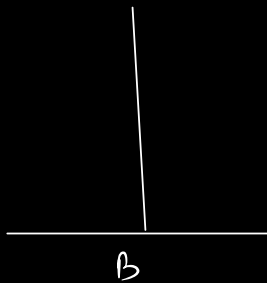
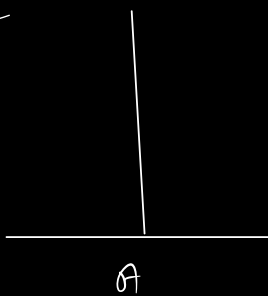
- Given three towers (A, B and C)
- There are N discs placed on tower A
- Move all discs from A to C.

⇒ Only 1 disc can be moved at a time

⇒ Larger disc can't be placed on smaller discs.

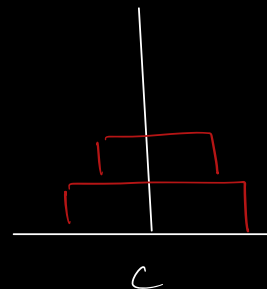
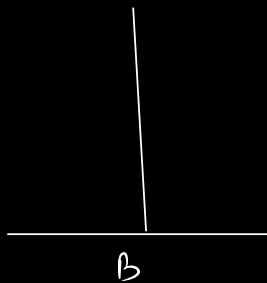
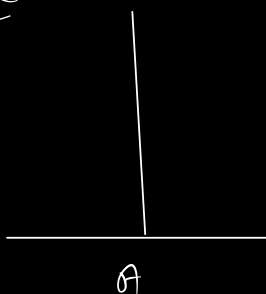
Print the movement of discs.

$n=1$



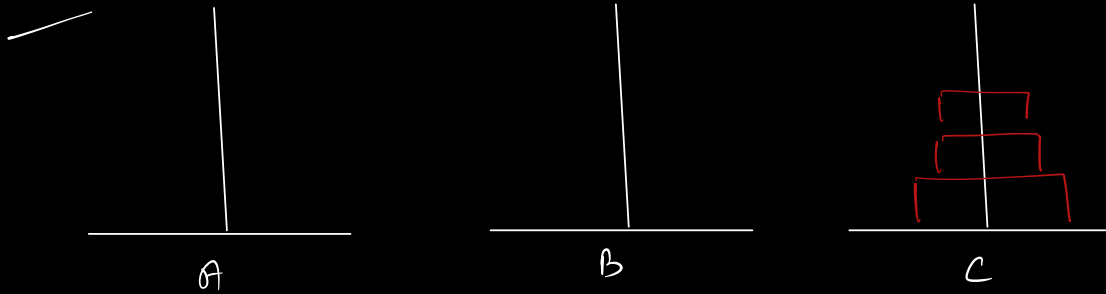
$A \rightarrow C$

$n=2$



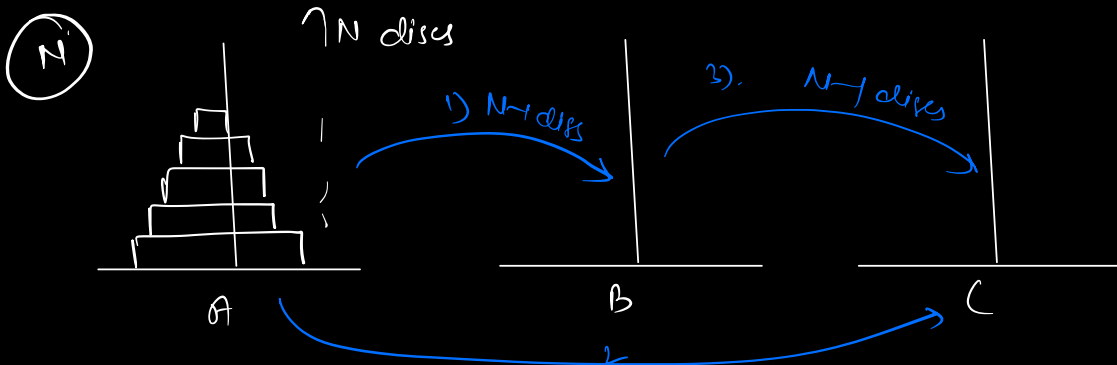
O/P { $A \rightarrow B$
 $A \rightarrow C$
 $B \rightarrow C$

N=3



O/P

- A → C
- A → B
- C → B
- A → C
- B → A
- B → C
- A → C



Main job: move N discs from A to C using B .

Step 1 → move $(N-1)$ discs from A to B using C

Step 2 → move largest disc from A to C ($A \rightarrow C$)

Step 3 → move $(N-1)$ discs from B to C using A

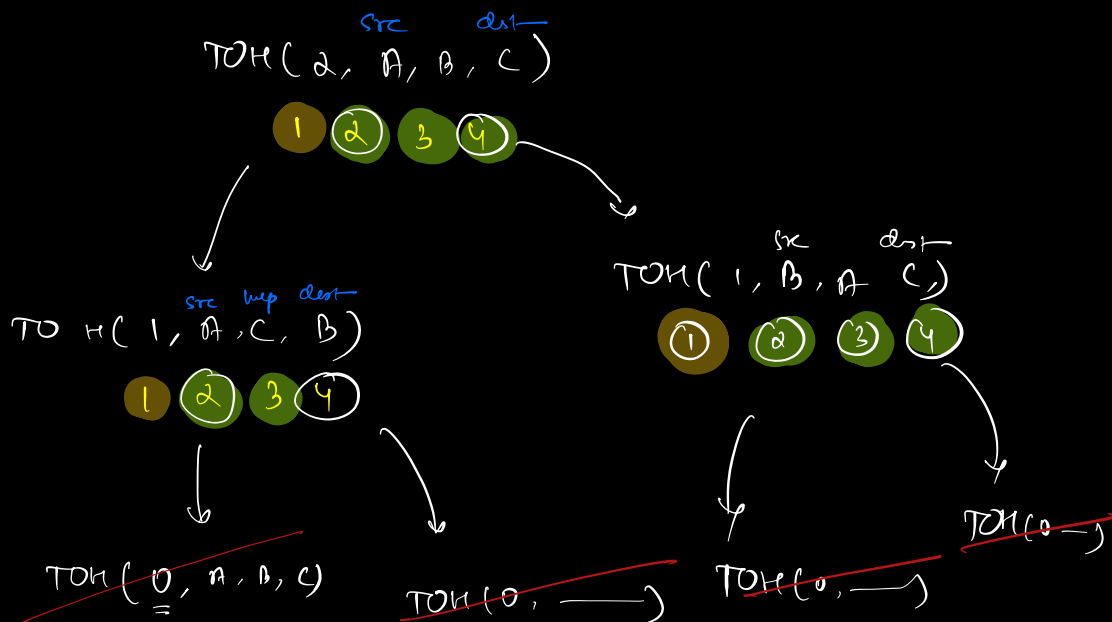
TC(N)

```

void TOH(int N, char src, char helper, char dest) {
    if (N == 0) {
        return;
    }
    // move N-1 discs from A to B using C
    TOH(N-1, src, dest, helper);
    print(src -> dest);
    // move N-1 discs from B to C using A
    TOH(N-1, helper, src, dest);
}

```

N=2



{

 A → B

 A → C

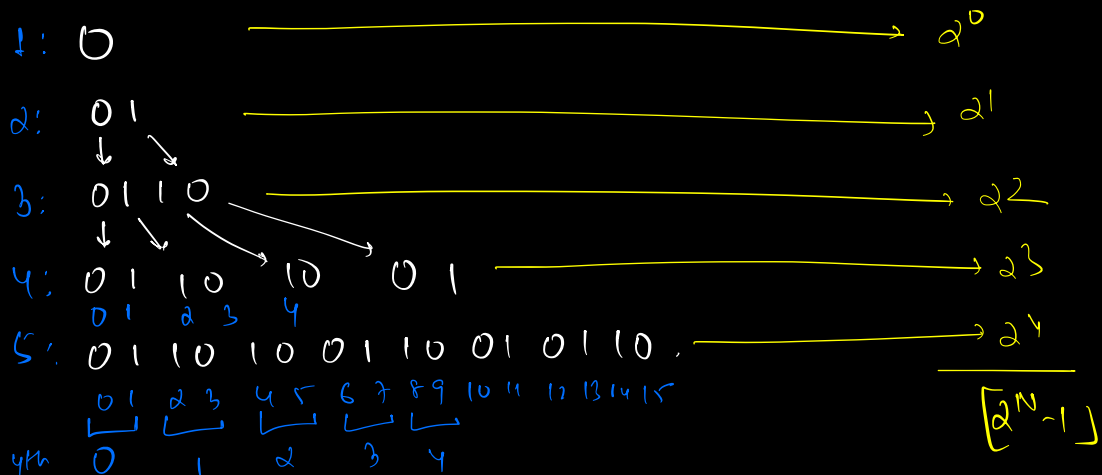
 B → C
 }

$$TCN) = 2TCN-1) + 1$$

$$\begin{cases} TC \Rightarrow O(2^N) \\ SC \Rightarrow O(N) \end{cases}$$

Q2. k^{th} Symbol

$$\begin{aligned} 0 &\rightarrow 01 \\ 1 &\rightarrow 10 \end{aligned}$$



return $\rightarrow$ k^{th} element of N^{th} row. [0-based indexing]

ex $\Rightarrow$ 4th element 4th row $\Rightarrow$ 1

6th element 5th row $\Rightarrow$ 0

idea!

generate N rows, then find k th element

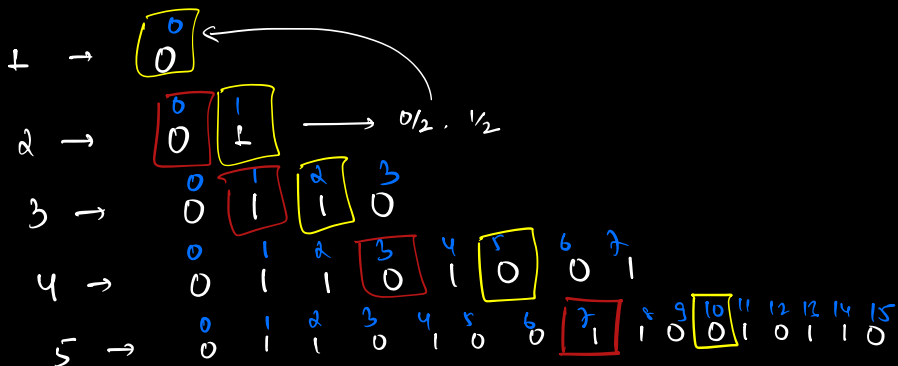
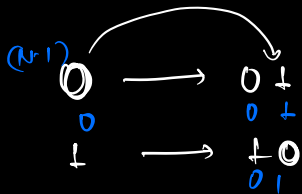
$$\begin{bmatrix} T_n & O(2^n) \\ SE_n & O(N) \end{bmatrix}$$

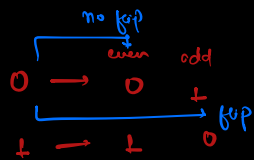
element $\Rightarrow$

N^{th} row	1^{th} element
$N-1^{\text{th}}$ row	$K/2^{\text{th}}$ element
$N-d^{\text{th}}$ row	$K/4^{\text{th}}$ element
$\vdots$	$\vdots$
0	0

N	k	elements
5	7	↓
4	3	0
3	1	1
2	0	0

N	k	element
5	10	0
4	5	0
3	2	1
2	1	1
1	0	0





1 → 0
 2 → 0 1
 3 → 0 1 2 3
 4 → 0 1 2 3 4 5 6 7
 0 1 1 0 1 0 0 1

pseudo

int symbol (N, k) {

if (N == 0 || k == 0)

return 0

int element = symbol(N-1, k/2);

if (k%2 == 0)

return element;

else

return ~element;

}

$$TC \Rightarrow O(\min(N, \log k))$$

↓
max^m k possible $\Rightarrow 2^N$

$$\max^m(\log_2 k) \rightarrow \log_2 2^N = N$$

$$O(\min(N, N))$$

TC	<u><u>$O(N)$</u></u>
SC $\Rightarrow$	$O(N)$

Q3. Gray code

Sequence of numbers where every 2 consecutive no. differ by only a single bit.

- always starts from 0.

$\Rightarrow$ Given N bit no., create the gray code sequence in decimal

$$\underline{N=1}$$

	dec
0	0
1	1

$$\underline{N=2}$$

	dec
00	0
01	1
11	3
10	2

Diagram showing a red arrow from 1 to 2 and a red arrow from 3 to 2, indicating a reflection or mapping.

$$\underline{N=2}$$

1	0
1	1
0	1
0	0

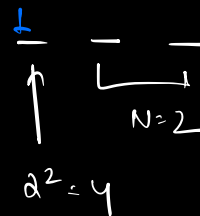
N=3

0	0	0
0	0	1
0	1	1
0	1	0
1	1	0
1	1	1
1	0	1
1	0	0

Deci

0
1
3
2
6
7
5
4

N=3



Op $\rightarrow$ 0 1 3 2 6 7 5 4

gray(N) $\downarrow$

generate gray(N-1)

insert gray(N-1) as it is.

and insert reverse gray(N-1) with appending 1 in the beginning.

gray(N) $\downarrow$
TCN)

generate gray(N-1) $\rightarrow$ list $\rightarrow$ TCN)

$\rightarrow$ ans $\leftarrow$ list

$\rightarrow$ add list[gray(N-1)] to ans

$\rightarrow$ add (2^{N-1}) to each element of gray(N-1) list and insert in reverse to ans list.

→ if $N=1$

gray code $\Rightarrow [0, 1]$

POSS → code for gray code

how? → $TC(N) = TC(N-1) + 2^{N-1}$
↓
expand

$$\begin{array}{l} TC \Rightarrow O(2^N) \\ SC \Rightarrow O(2^N) \end{array}$$

↓
recursion stack $\rightarrow O(N)$